

HAI819I – Moteur de Jeux

Rapport de projet

Léo Hafdane – Matis Bégot

[*https://github.com/Leottaro/HAI819I_moteur_jeux*](https://github.com/Leottaro/HAI819I_moteur_jeux)

27 mai 2026

M1 Informatique – IMAGINE
Faculté des Sciences
Université de Montpellier.



Table des matières

1	Résumé du projet	1
2	Bases essentielles	1
3	Spécialisation	2
3.1	Monde 3D voxelisé	2
3.1.1	Spécialisation de la détection de collisions	2
3.1.2	Spécialisation de l’affichage	2
3.2	Mesure de temps globale avec système de ticks	2
3.3	Cycle jour/nuit	3
3.4	Pierre de Lit	3
4	Interactions	3
5	Rendu	4
6	Prototype	4
6.1	Génération	4
6.1.1	Debug	4
6.1.2	Superflat	4
6.1.3	Surmonde	5
7	Outils développés	5
7.1	Conteneur de stockage	5
7.2	Générateur d’Atlas et de Texture2DArray	5
7.3	Hot reload des shader et textures	6
7.4	Gestionnaire d’inputs	6
8	Liens	6

1 Résumé du projet

Nous avons décidé de faire un moteur de jeu Minecraft-like, c'est-à-dire un jeu dont l'environnement est intégralement composé de blocs (*voxels*) sur lesquels sont plaqués des textures. Notre moteur permet de gérer ce monde par segments (*chunks*) de $N \times N \times N$ blocs. Le moteur permet également de gérer des entités qui peuvent interagir avec le monde et sont soumises aux lois de la physique, le tout en visant des performances temps réel. Nous avons multithreadé le moteur, séparant le processus principal de rendu et le processus du monde.

2 Bases essentielles

Notre moteur est capable de :

- **Charger des maillages** : Il possède ce qu'il faut pour charger et afficher des maillages.
- **Graphe de scène** : Le moteur possède un système classique de Graphe de scène mais le moteur visant à gérer le monde par Chunk, celui-ci n'est pas utilisé.
- **Moteur physique** : Un moteur physique est implémenté, il impacte essentiellement les entités et prend en compte les forces de gravité, de flottaison et de traînée.
- **Détection de collisions** : Un système de détection de collision continue entité/monde ainsi que des réactions (réstitution, friction, friction statique) à ces dernières est implémenté.
- **PBR** : Le PBR est implémenté. Il peut supporter toutes les composantes classiques du PBR.
- **ECS** : Toutes nos entités sont gérées par un ECS (*Entity Component System*). Pour le rendre le plus efficace possible, les composants, systèmes et types d'entités sont tous définis à la compilation. De plus, nous avons fait le choix de définir un nombre d'entité maximum qui est de 2^{20} (volontairement très haut) pour une mémoire statique à $200MiB$ pour tous les composants.

3 Spécialisation

3.1 Monde 3D voxelisé

3.1.1 Spécialisation de la détection de collisions

Pour gérer ce monde entièrement voxelisé, il est nécessaire de le ségmenter par Chunks. Cette ségmentation nous permet de gérer les mises à jour ainsi que l’affichage de manière localisée afin de limiter les calculs inutiles. Le fait que le monde soit voxelisé nous aide également grandement pour la détection de collisions. En effet, les calculs de AABB y sont les plus efficaces. De plus, un simple algorithme de franchissement de voxel¹ permet une *broad phase* extrêmement rapide.

3.1.2 Spécialisation de l’affichage

À chaque mise à jour d’un chunk, son maillage est régénéré comme étant un ensemble des faces de ses blocs. Nous avons implémenté du culling qui ne compte pas les faces des blocks solides colés.

Nous avons fait le choix de ne pas sauvegarder ces données d’affichages. En effet, garder en mémoire l’ensemble de ses triangles aurait à priori été très lourd, nous avons privilégié un gain en mémoire pour une perte CPU. Cependant, puisqu’on ne sauvegarde pas les différents triangles et que nous avons des blocs translucides, le chunk doit régénérer son maillage entier quand la caméra se déplace pour remettre ses triangles translucide dans l’ordre, et cela semble trop lourd. Il serait nécessaire d’implémenter diverses solutions et de faire des tests poussés afin de savoir quelle est la bonne.

3.2 Mesure de temps globale avec système de ticks

Pour mesurer le temps dans notre monde, nous avons rajouté un système de ticks intégré au rendu du monde dans un thread. Ce thread est conçu pour vérifier qu’il est temps d’avancer d’une tick, si ce n’est pas encore le cas, le thread s’endort et se réveille au moment voulu. Ce système peut s’avérer utile pour beaucoup d’autres fonctionnalités du jeu comme expliqué ci-dessous et pourrait servir à bien d’autres.

¹https://www.researchgate.net/publication/2611491_A_Fast_Voxel_Traversal_Algorithm_for_Ray_Tracing

3.3 Cycle jour/nuit

Le cycle jour/nuit utilise cette fonctionnalité pour permettre de rendre le monde plus "vivant" et permet de mieux tirer partie de l'éclairage PBR et du système d'ombres en shadow map. Pour calculer l'angle du soleil, on calcule l'angle du soleil sur l'axe est/ouest en fonction de l'heure de la journée (en ticks), on prend également son angle sur l'axe nord/sud pour aussi varier les saisons. Une fois tout ça intégré dans un `glm::vec2`, il faut transformer ça en vecteur unitaire représentant l'angle du soleil sur n'importe quelle surface, pour cela il a fallu utiliser une représentation assez spéciale, le *n-vector*². Cette formule nous donne à partir des angles du soleil le vecteur correspondant :

$$\begin{bmatrix} \cos latitude \times \cos longitude \\ \cos latitude \times \sin longitude \\ \sin latitude \end{bmatrix}$$

Une fois envoyé dans le shader, ce vecteur fait office de lumière à part entière. Cette lumière est spéciale parce que compte tenu de sa distance à notre scène, sa position n'est pas requise pour connaître son angle et il n'a pas besoin d'être calculé pour chaque fragment, ce qui rend le calcul plus efficace qu'une lumière classique. Pour finir, la couleur du ciel et du soleil change en fonction de l'heure de la journée.

3.4 Pierre de Lit

La pierre de lit est un bloc à la fois original³ et à la fois révolutionnaire de notre moteur. En effet, une fois placé, il est impossible de le retirer : un message sera affiché dans les logs pour en informer le joueur. Il permet aussi de ne pas permettre aux joueurs de tomber à travers le monde. De plus, contrairement aux copies de notre moteur, il n'existe pas encore de bugs permettant de passer à travers ou la casser.

4 Interactions

En ce qui concerne le HUD, nous avons de simples fenêtres ImGui⁴ pour interagir avec les paramètres de la caméra, du monde, ainsi que de l'entité actuellement contrôlée. Les entités peuvent également interagir avec le monde : elles lui soumettent des changements à faire, puis le monde les exécute une fois par tick.

²<https://en.wikipedia.org/wiki/N-vector>

³Ce bloc est tellement original que le Minecraft Wiki y a dédié un article en traduisant son nom en anglais : <https://minecraft.wiki/w/Bedrock>

⁴<https://github.com/ocornut/imgui>

5 Rendu

Pour faire le rendu de notre scène, nous avons choisi (et aussi un peu parce que c'était les consignes) de faire un rendu PBR (*Physically Based Rendering*). Pour utiliser toutes les composantes du PBR, nous avons 3 textures par bloc.

La première texture est l'*albedo*, elle représente la couleur que notre texture doit avoir, son canal alpha représente le niveau de transparence de notre bloc.

La deuxième est la *normal map*, cette texture permet de modifier la normale de la surface pour donner une impression de relief sans toucher à la géométrie de notre scène, le canal alpha permet de mixer entre la normale de la surface et notre normale personnalisée.

La dernière texture est la spéculaire, ses canaux RGBA représentent respectivement la rugosité, la métallicité, le facteur de Fresnel et l'occlusion ambiante (bien que ce dernier effet soit globalement négligeable étant donné la faible résolution des textures).

Nous avons commencé à travailler sur du post processing, notamment sur du SSAO, du brouillard de distance et un effet quand la caméra est dans l'eau. Nous n'avons malheureusement pas eu le temps de fini mais c'est sur la branche `post_processing`

6 Prototype

6.1 Génération

Notre prototype possède 3 types de génération de monde dont certains sont seedés. Les seeds sont générées par un champ que l'utilisateur peut modifier, la seed sera soit castée en nombre si l'utilisateur a effectivement entré un nombre, soit hashée si c'est ce n'est pas castable. (Cela permet de s'amuser et de voir à quoi ressemble un monde qui porte le nom de son chien/sa voiture/ la marque de sa plaque de cuisson.)

6.1.1 Debug

Un monde plat avec une couche de *Pierre de Lit* à $y = 0$ et une couche sur laquelle tous les blocs sont représentés de façon itérative pour tester le rendu de chaque. Ce monde ne propose aucune variation avec les seeds

6.1.2 Superflat

Un monde similaire au monde **Debug**, mais où la couche du dessus est composée d'un seul bloc. De plus, il est possible de changer à l'aide de la seed le type de bloc de cette surface.

6.1.3 Surmonde

Le clou du spectacle : une dimension réaliste, générée procéduralement à l'aide d'une heightmap (à l'aide d'un bruit de perlin, généré par la bibliothèque STB). Le surmonde⁵ peut prendre une seed en entrée qui sera utilisée par STB pour régénérer un monde par dessus. De plus, il est possible de générer des structures à la surface de notre monde, pour l'instant leur apparition est régie seulement par une variable aléatoire tirée à chaque bloc de surface.

7 Outils développés

7.1 Conteneur de stockage

Pour essayer d'améliorer la continuité en mémoire de nos Chunks, nous avons créé un conteneur qui en alloue plusieurs d'un coup et fais le lien entre un Chunk et sa position. Un Chunk étant très grand, nous évitons complètement les réallocations tout en ayant un peu plus de continuité qu'avec une simple map.

7.2 Générateur d'Atlas et de Texture2DArray

Pour pouvoir rendre les Chunks tout en gardant des textures individuelles pour les blocks, nous avons dans un premier temps implémenté un générateur d'atlas automatique. Dans notre programme nous renseignons les différentes textures disponibles (face) ainsi que leur nom attendu. Ensuite, pour chaque type de bloc, nous avons renseigné leur texture pour chaque face. Enfin au début du programme, notre moteur compile toutes ces informations pour générer l'atlas voulue.

En ajoutant des blocks, nous nous sommes rendu compte que des problèmes apparaissaient au fur et à mesure que l'atlas grandissait. Les résoudre auraient nécessité d'ajouter un padding mais cela n'était pas très élégant. Nous avons donc décidé de générer des Texture2DArray qui nous ont permis d'éliminer les problèmes de padding et de rendre les MipMap possibles.

Coté texture, nous avons implémenté une variation aléatoire basée sur la position du cube. Nous pouvons facilement spécifier quelles textures sont "tournables", inversable en X ou en Y.

⁵Encore une idée tellement originale que le Minecraft Wiki se l'est appropriée et l'a anglicisée : <https://minecraft.wiki/w/Overworld>

7.3 Hot reload des shader et textures

Pour rendre la programmation plus facile, nous avons utilisés des FileWatchers ⁶ qui, lorsqu'ils détectent un changement dans les dossiers des shaders ou des textures, recharges et recompile les shader et atlas.

7.4 Gestionnaire d'inputs

Pour simplifier la gestion des inputs clavier et souris, nous avons créer un gestionnaire auquel on peut ajouter des callbacks, modifier les touches allouées, etc...

8 Liens

- git : https://github.com/Leottaro/HAI819I_moteur_jeux
- rapport : <https://plmlatex.math.cnrs.fr/read/dkdtdqfycjsz>
- vidéo : <https://youtu.be/30I92G2UMks>

⁶<https://github.com/ThomasMonkman/filewatch>